O que é?

O projeto é um backend de um sistema de gerenciamento de chamados, feito em haskell, tentando utilizar boa parte das técnicas de desenvolvimento guiado a tipos vistas em aula. O backend foi separado em 4 principais áreas, API, Application, Domain e Infrastructure.

- API: Tudo relacionado à API, HTTP, JSON, e o Servant
- Application: Funções do sistema, como por exemplo registrar um usuário
- Domain: No domain está as principais regras de negócio, boa parte das técnicas foram utilizadas aqui
- Infrastrucutre: Para as implementações que relacionadas ao banco de dados

Link para explicação do projeto: https://youtu.be/ATEvxjfBo-4

Escolha de bibliotecas

Durante o desenvolvimento, foi necessário escolher algumas bibliotecas para resolver alguns problemas em específico.

- bcrypt: O bcrypt foi escolhido por causa do escopo que ele trabalha, que é com a encriptação de senhas
- uuid: O uuid foi adicionado para criação de identificadores únicos não sequenciais, por boa prática
- persistent: O persistent foi escolhido porque ele suportava o SQLite, e conseguia manter a garantia dos tipos
- servant: O servant foi escolhido por conta da sintaxe simples
- aeson: Foi necessário para trabalhar com json
- wai, warp: Só para dar run no servidor
- http-api-data: Para colocar os tipos nos dados que vem da url

Dificuldades, surpresas e destaques

As maiores dificuldades ao longo do desenvolvimento foram relacionados às bibliotecas, em alguns casos não consegui fazer o persistent exportar os campos que ele deveria gerar, tanto que eu gostaria de ter feito uma consulta SQL em findUserEntityById, mas não consegui por conta disso. Além disso, tive dificuldade na criação de testes, principalmente com os banco de dados. Eu queria ter feito um banco de dados de teste, mas não consegui, porque eu não consegui abstrair o suficiente para que o nome do banco de dados fosse dado em apenas um lugar.

Sobre o projeto em geral, eu subestimei um pouco o tempo. A ideia inicial era fazer um sistema completo, com front-end, até cheguei a fazer o design, mas não foi possível realmente fazer o código.

Sobre os destaques do projeto, é que foi possível utilziar boa parte das técnicas aprendidas, ADTs (Cargos dos usuários), GADTs (Criação de Tickets), DataKinds + Phanthom Types (Estados de validação), Type Families (transição dos tickets). Não achei uma oportunidade para utilizar tipos existenciais, por exemplo.

Como rodar?

Para rodar o servidor, **stack run**, para visualização do banco de dados, recomendo o DB Browser for SQLite, e para testar a API, recomendo o Insomnia